Lab 5: Mixing with C

- **Link**
  - Video
  - HackMD

- **Basic** (50%)
  - **Description**
    The program **main.c** will call the function gcd to find the greatest common divisor of two input numbers. Please complete the gcd function using PIC18F assembly language. The inputs of the function will be two 8-bit unsigned chars, a and b. The return value should be their greatest common divisor as an 8-bit unsigned char. The result should be stored in an unsigned char variable named ans. Please be aware that you will need two files, **main.c** and **gcd.asm**, to fulfill this requirement.
    Function signature:
    ```
    extern unsigned char gcd(unsigned char a, unsigned char b);
    ```

  - **Example**
    gcd(5, 10) = 5
    gcd(252, 105) = 21
    gcd(17, 5) = 1

  - **Standard of grading**
    1. Mixing with C: Implement the function in assembly and call it from the main C program.
    2. The function name and the variable name in main.c must be the same as described.
    3. Please show the result in the Watches window.
    4. All input test cases will fall between 1 and 255.

- **Advanced** (30%)
  - **Description**
    The program **main.c** will call an assembly function named **count_primes** to determine the number of prime numbers within a given inclusive interval. Your task is to implement the **count_primes** function using PIC18F assembly language. The function receives two 16-bit unsigned integers, `n` and `m`, where `n < m`. It must count all prime numbers `p` satisfying `n ≤ p ≤ m`. The resulting count must be returned to the C program and stored in an `unsigned int` variable named `ans`.

    You must use two source files, `main.c` and `count_primes.S`, to complete this task. Since the grading system evaluates both correctness and execution time, your implementation should avoid unnecessary computation and must follow the interface and implementation requirements specified on HackMD.

- **Example**
  1. count_primes(2, 3) = 2
  2. count_primes(114, 514) = 67
  3. count_primes(1, 65535) = 6542

- **Standard of grading**
  1. Your implementation will be evaluated using an online judge with multiple test cases.
  2. Both the correctness of the returned result and the program execution time, measured by cycle count, will be considered.
  3. Your implementation must therefore follow the specifications provided on HackMD exactly and complete all test cases within the required execution-time limit.

- **Hard** (20%)
  - **Description**

    The C program `main.c` will call an assembly function named `evaluate` to evaluate an arithmetic expression containing four 16-bit unsigned integers and three operators. Your task is to implement the `evaluate` function in PIC18F assembly language.

    The expression has the following form:
    `a op1 b op2 c op3 d`
    where `a`, `b`, `c`, and `d` are 16-bit unsigned integers, and `op1`, `op2`, and `op3` are 8-bit character values representing the operators. The supported operators are `+`, `-`, and `*`.

    The function must evaluate the expression according to standard arithmetic precedence. Multiplication (`*`) has higher precedence than addition (`+`) and subtraction (`-`). Operators with the same precedence must be evaluated from left to right. For example,

    `2 + 3 * 4 - 5`

    must be evaluated as

    `2 + (3 * 4) - 5 = 9`.

    The final result must be returned by the assembly function and stored in an `unsigned int` variable named `ans` in `main.c`. You must use two source files, `main.c` and `evaluate.asm`, to complete this task.

    The function declaration in `main.c` must be:
    `extern unsigned int evaluate(unsigned int a, char op1, unsigned int b, char op2, unsigned int c, char op3, unsigned int d);`

    All numerical inputs are 16-bit unsigned integers. All

intermediate results and the final result are guaranteed to remain within the 16-bit unsigned range from 0 to 65535. No test case will require representing a negative intermediate or final result.

- **Example**
  evaluate(5, '+', 3, '-', 2, '+', 1) = 7
  evaluate(2, '+', 3, '*', 4, '-', 5) = 9
  evaluate(12, '+', 34, '*', 2, '*', 3) = 216

- **Standard of grading**
  1. The evaluate function must be implemented in PIC18F assembly language and called correctly from the C program.
  2. The function name, declaration, and the variable name ans must follow the specification above.
  3. The expression must be evaluated using the specified operator-precedence and left-to-right rules, and the correct final result must be returned and displayed in the Watches window during the demonstration.